

PROJECT REPORT

Mini Search Engine

A Console-Based Java Application for Indexing and Searching Text Files

Field	Details
Student Name	D Jaya Deepak Reddy
Registration / Roll No.	25BMR10007
Course / Subject	VITyarthi Project
Programming Language	Java
Submission Date	18 September 2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my faculty members for their guidance and support throughout the development of this project. Their teaching helped me connect core Java concepts such as collections, file handling and exception handling with a practical, working application.

I also acknowledge the official Java Platform documentation, which was referred to while designing the indexing and search logic used in this project. Finally, I thank my classmates and family for their encouragement during the completion of this project.

ABSTRACT

The Mini Search Engine is a console-based Java application that indexes every .txt file inside a chosen folder and allows a user to search across all of them by keyword. It supports single-keyword and multi-keyword queries, case-insensitive matching, ranked results based on match count, and a running search history for the current session.

The application is built around four cooperating classes: `IndexedFile`, which represents one file's name and content; `SearchResult`, which represents the outcome of searching a single file; `SearchEngine`, which performs indexing and searching; and `MiniSearchEngine`, which drives the command-line interface. Java's NIO file APIs are used to read the folder and its files, `ArrayList` is used to store indexed files, matched lines, results and history, and results are ranked using a `Comparator` that sorts by descending match count.

The project was tested against normal keyword searches, multi-keyword searches, case-insensitive input, queries with no matches, empty input, and error conditions such as a missing or empty documents folder. All cases were verified to behave correctly without the program crashing, confirming that the application meets its intended requirements.

TABLE OF CONTENTS

Acknowledgement.....	2
Abstract	2
1. Introduction	4
2. Problem Statement	4
3. Objectives.....	4
4. Scope and Users	4
5. Requirements Analysis.....	5
5.1 Functional Requirements	5
5.2 Non-Functional Requirements.....	5
6. System Design and Architecture	6
7. Design and Key Java Concepts Used.....	7
8. Module Description	9
9. Implementation Details	9
10. Design Diagrams	10
11. Testing and Results	11
12. Limitations.....	11
13. Future Enhancements.....	12
14. Conclusion	12
References	12
15. Sample Output Screenshots	13

1. INTRODUCTION

Searching manually through a folder of text files for a particular word or phrase becomes slow and error-prone as the number of files grows. The Mini Search Engine was developed to automate this task using core Java, without relying on any external search library or database.

The application reads every .txt file placed inside a documents folder into memory when it starts, and then repeatedly accepts a keyword or phrase from the user through the command line. It reports which files contain the keyword, how many lines matched in each file, and the actual matching lines, ranked so that the most relevant file appears first.

The project is implemented in a single Java source file containing four classes, keeping the submission simple to compile and run while still demonstrating good separation of concerns between file indexing, search logic, and the command-line interface.

2. PROBLEM STATEMENT

Manually opening and scanning multiple text files to locate a keyword does not scale and is prone to human error, especially when a folder contains many documents. A small search utility is therefore needed that can index a folder's contents once and then answer repeated keyword queries quickly and consistently.

The project addresses the following problems:

- How can every .txt file in a folder be read and held in memory for repeated searching?
- How can a query be matched against file contents regardless of letter case?
- How can a query consisting of multiple words be searched at once?
- How can matching files be ranked so the most relevant result appears first?
- How can previously run searches be recalled during the same session?
- How can the program avoid crashing on empty input or a missing folder?

3. OBJECTIVES

The primary objective is to build a functional, console-based text search utility in core Java that demonstrates sound use of classes, collections and file handling. The specific objectives are:

- To index every .txt file inside a given folder into memory.
- To support single-keyword and multi-keyword (OR) search across all indexed files.
- To make search case-insensitive.
- To display, for each matching file, the number of matches and the matching lines.
- To rank files by descending match count.
- To maintain a search history for the running session.
- To handle invalid input and folder errors gracefully, without the program crashing.

4. SCOPE AND USERS

The application is designed for academic demonstration and small-scale local use. It covers folder indexing, keyword search, multi-keyword search, ranking, and search history, all through a command-line interface.

The system has a single category of user: a person running the program from a terminal who wants to search a local folder of text files. There is no login, no network access and no external service dependency - the program only reads local files supplied by the user.

The present version does not index file formats other than .txt, does not search file names or subfolders, and does not persist search history between separate runs of the program.

5. REQUIREMENTS ANALYSIS

5.1 Functional Requirements

ID	Requirement
FR-01	The system shall read every .txt file directly inside a given folder.
FR-02	The system shall reject startup if the given folder does not exist.
FR-03	The system shall accept a keyword or phrase typed by the user.
FR-04	The system shall treat search input as case-insensitive.
FR-05	The system shall support multiple space-separated keywords in one query, matching a line if it contains any of them.
FR-06	The system shall report, per matching file, the total match count and the matching lines.
FR-07	The system shall rank files by match count, highest first.
FR-08	The system shall record every search query in a session history.
FR-09	The system shall display the search history on request.
FR-10	The system shall reject empty input without crashing.
FR-11	The system shall exit cleanly on the 'exit' command.

5.2 Non-Functional Requirements

Requirement	Implementation in the Project
Usability	Clear numbered results, simple prompts, and repeated re-prompting on invalid input.
Reliability	Checked-exception handling around folder access; empty-folder and no-match cases handled without errors.
Performance	All files are read into memory once at startup, so repeated searches only scan in-memory data.
Maintainability	Indexing, result modelling and search logic are separated into distinct classes.

Portability	Uses only standard Java libraries (java.nio.file, java.util) with no external dependencies.
-------------	---

6. SYSTEM DESIGN AND ARCHITECTURE

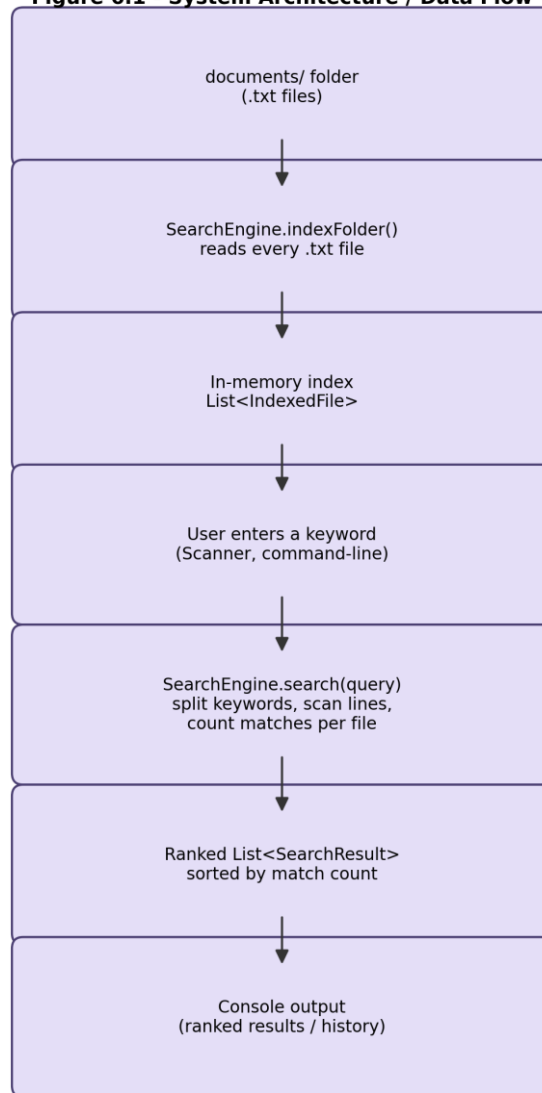
The system follows a simple linear pipeline rather than a multi-layer architecture, which suits the scope of a console search utility.

Indexing stage: `SearchEngine.indexFolder()` locates every `.txt` file in the documents folder using `Files.newDirectoryStream()` and reads each one's lines with `Files.readAllLines()`, wrapping each file as an `IndexedFile` stored in an in-memory `ArrayList`.

Search stage: `SearchEngine.search()` splits the user's query into keywords, lower-cases both the query and each stored line, and scans every indexed file's lines for a match. A `SearchResult` is built for every file that has at least one matching line.

Ranking and output stage: results are sorted using `Comparator.comparingInt(SearchResult::getMatchCount).reversed()` before `MiniSearchEngine` prints them to the console.

Figure 6.1 - System Architecture / Data Flow



7. DESIGN AND KEY JAVA CONCEPTS USED

The project intentionally keeps its object model simple, matching the scope of a single-purpose console tool. The following Java and design concepts are demonstrated:

7.1 Encapsulation

IndexedFile and SearchResult keep their fields private and expose them only through getter methods, preventing external code from directly modifying a file's stored lines or a result's match count.

7.2 Separation of Concerns

Responsibilities are divided across four classes: IndexedFile models one file, SearchResult models one search outcome, SearchEngine owns indexing and search logic, and MiniSearchEngine owns only the command-line interaction loop.

7.3 Collections

ArrayList is used throughout: to hold the list of indexed files, the lines of each file, the matching lines of each result, the list of results, and the session's search history.

7.4 File Handling (NIO)

java.nio.file.Path, Files and DirectoryStream are used to check that the folder exists, filter for .txt files only, and read file contents line by line.

7.5 Exception Handling

indexFolder() throws a checked IOException when the folder is missing, which MiniSearchEngine.main() catches and reports with a clear message instead of letting the program crash with a stack trace.

7.6 Sorting

java.util.Comparator is used to rank SearchResult objects by descending match count before they are displayed.

Note: this project does not use inheritance, interfaces or polymorphism, since a single-purpose search tool with one kind of file and one kind of result does not call for a class hierarchy. Encapsulation, collections and clean method decomposition are used instead to keep the design straightforward and appropriate to the problem size.

8. MODULE DESCRIPTION

8.1 Indexing Module

Implemented in `SearchEngine.indexFolder()`. Validates that the target folder exists, clears any previous index, then reads every `.txt` file's lines into an `IndexedFile` and stores it in the index list.

8.2 Search Module

Implemented in `SearchEngine.search()`. Splits the query into lower-cased keywords, scans every indexed file's lines, and counts a line as a match if it contains any one of the keywords. Builds a `SearchResult` for every file with at least one match.

8.3 Ranking Module

Part of the same `search()` method: once results are collected, they are sorted by match count in descending order so the most relevant file is shown first.

8.4 History Module

Every query passed to `search()` is appended to a `searchHistory` list, regardless of whether it produced matches. The history command lists these queries in order.

8.5 Command-Line Interface Module

Implemented in `MiniSearchEngine.main()` and its helper methods `printResults()` and `printHistory()`. Runs a loop that reads input, recognises the history and exit commands, and otherwise treats the input as a search query.

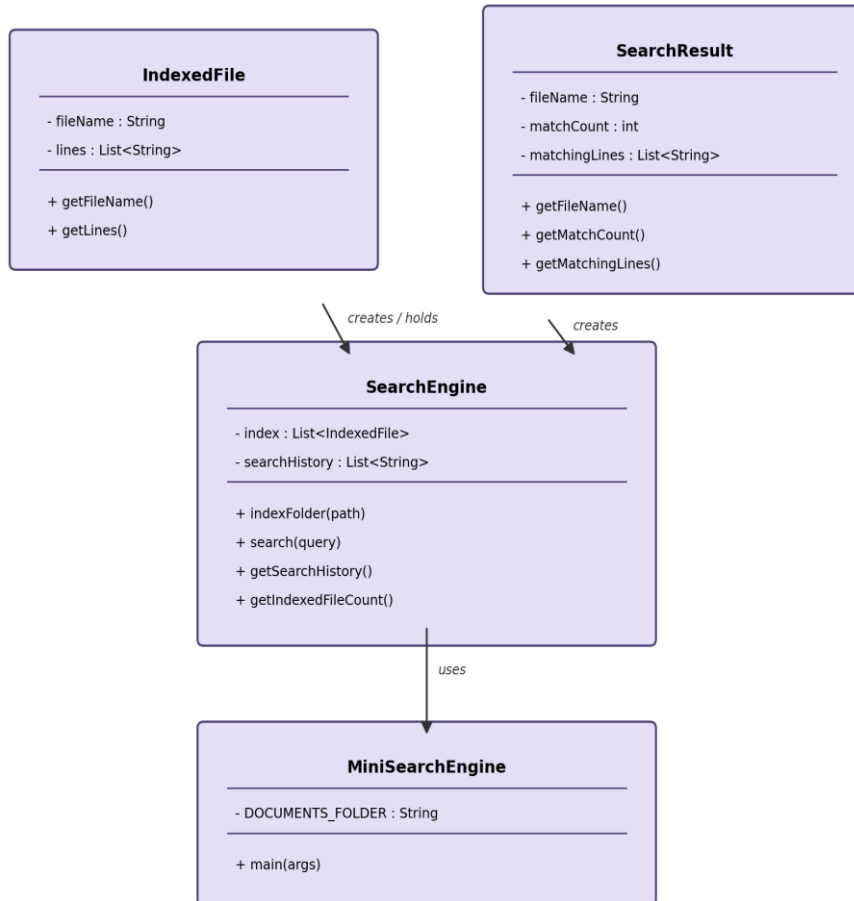
9. IMPLEMENTATION DETAILS

Item	Implementation
Entry point class	<code>MiniSearchEngine</code>
Supporting classes	<code>IndexedFile</code> , <code>SearchResult</code> , <code>SearchEngine</code>
Default documents folder	<code>documents</code> (relative to the working directory)
Collections used	<code>ArrayList<IndexedFile></code> , <code>ArrayList<String></code> , <code>ArrayList<SearchResult></code>
File reading API	<code>java.nio.file.Files.readAllLines()</code> , <code>DirectoryStream</code> with a <code>*.txt</code> filter
Case handling	Both query and file lines converted with <code>String.toLowerCase()</code>
Multi-keyword logic	Query split on whitespace; a line matches if it contains any keyword (logical OR)
Ranking	<code>Comparator.comparingInt(SearchResult::getMatchCount).reversed()</code>
Reserved commands	<code>history</code> (shows past queries), <code>exit</code> (ends the program)

10. DESIGN DIAGRAMS

10.1 Class Diagram

Figure 10.2 - Class Diagram (Mini Search Engine)



11. TESTING AND RESULTS

Testing covered normal keyword searches as well as edge cases: multi-keyword queries, case sensitivity, no-match queries, empty input, and folder-related errors. All test cases were run against the compiled program (javac / java, JDK 21).

Test ID	Test Scenario	Expected / Actual Output
T01	Search a single keyword present in one file (e.g. bitcoin)	That file is ranked first with its match count; other files containing the word appear below it. Verified - see Figure 15.1.
T02	Search two keywords at once (e.g. ocean travel)	Any line containing either word counts as a match; both relevant files appear, ranked by match count. Verified - see Figure 15.2.
T03	Search using a different letter case (e.g. HEALTH)	Produces identical results to the lowercase query, confirming case-insensitive matching. Verified - see Figure 15.3.
T04	Search a keyword that matches nothing (e.g. zzzznotfound)	Program prints 'No matches found.' without error. Verified - see Figure 15.3.
T05	Press Enter with no input	Program prints 'Please enter a keyword.' and re-prompts instead of searching. Verified - see Figure 15.4.
T06	Type 'history' after running several searches	All prior queries are listed in the order they were run. Verified - see Figure 15.4.
T07	Type 'exit'	Program prints 'Goodbye!' and terminates the loop cleanly. Verified - see Figure 15.4.
T08	Run the program with no documents folder present	Program prints 'Error while indexing files: Folder not found: documents' and exits without a stack trace.
T09	Run the program with an empty documents folder (no .txt files)	Program starts normally and reports 'Indexed 0 file(s)'; any subsequent search reports no matches.

All nine scenarios behaved as expected. In particular, invalid or unusual input (empty query, no-match query, missing folder, empty folder) was handled through existing validation and exception handling rather than causing the program to terminate unexpectedly.

12. LIMITATIONS

The application is an academic tool rather than a production search engine. Its limitations include:

- Only files with a .txt extension directly inside the documents folder are indexed; subfolders are not scanned.
- Multi-keyword search is an OR match on individual words, not exact-phrase matching.
- Search history exists only for the current run and is not saved between sessions.
- The folder to index is fixed to a constant (documents) rather than being configurable at runtime.

- There is no support for regular expressions, stemming, or ignoring common stop words.
- Matching is line-based, so a keyword split across two lines will not be found.

13. FUTURE ENHANCEMENTS

Several improvements could extend the project beyond its current academic scope:

- Add an exact-phrase search mode alongside the current OR-based multi-keyword search.
- Allow the user to type the folder path at runtime instead of using a fixed constant.
- Recursively index .txt files inside subfolders.
- Persist search history to a file so it survives between runs.
- Highlight the matched keyword within each printed line.
- Add a simple GUI or web front end using the same SearchEngine logic.

14. CONCLUSION

The Mini Search Engine meets all of its stated objectives: it indexes a folder of text files, supports single- and multi-keyword case-insensitive search, ranks results by relevance, maintains a session search history, and handles invalid input and folder errors without crashing.

The project demonstrates practical use of core Java concepts including collections, NIO file handling, exception handling, and sorting with a Comparator, organised across a small number of classes with clearly separated responsibilities. Its documented limitations point to realistic directions for further development, such as phrase search, persistent history, and a graphical interface.

REFERENCES

[1] Java Platform, Standard Edition 21 API Documentation -
<https://docs.oracle.com/en/java/javase/21/docs/api/>

[2] Java Files Class Documentation -
<https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/nio/file/Files.html>

[3] Java ArrayList Class Documentation -
<https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/ArrayList.html>

[4] Java Comparator Interface Documentation -
<https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Comparator.html>

15. SAMPLE OUTPUT SCREENSHOTS

The following screenshots were captured from an actual execution of MiniSearchEngine.java on the student's machine, compiled and run with javac / java (JDK 21). The run demonstrates bitcoin single-keyword search, ocean travel multi-keyword search, case-insensitive search, a no-match query, empty-input handling, the history command, and a clean exit.

15.1 Single-Keyword Search - "bitcoin"

```
Mini Search Engine
=====
Indexed 9 file(s) from 'documents'.

Enter search keyword (or 'history' to view past searches, 'exit' to quit): bitcoin

Search Results
=====
1. bitcoin.txt
  Matches: 10
  Matching lines:
    - Bitcoin is a decentralized digital currency created in 2009.
    - Bitcoin transactions are recorded on a public ledger called the blockchain.
    - The total supply of bitcoin is capped at 21 million coins.
    - Bitcoin mining uses computing power to validate transactions.
    - Many investors treat bitcoin as a store of value like digital gold.
    - Bitcoin price is known for being extremely volatile.
    - Wallets are used to store and manage bitcoin securely.
    - Bitcoin was created by an anonymous person or group called Satoshi Nakamoto.
    - Exchanges allow users to buy and sell bitcoin for regular currency.
    - Bitcoin popularized the idea of cryptocurrency worldwide.
2. finance.txt
  Matches: 2
  Matching lines:
    - Ethereum is another popular cryptocurrency besides bitcoin.
    - Some companies now accept cryptocurrency like bitcoin as payment.

Enter search keyword (or 'history' to view past searches, 'exit' to quit): |
```

Figure 15.1 - Searching for "bitcoin": bitcoin.txt ranked first, finance.txt ranked second

15.2 Multi-Keyword Search - "ocean travel"

```
Enter search keyword (or 'history' to view past searches, 'exit' to quit): ocean travel
```

```
Search Results
```

```
=====
```

```
1. ocean.txt
```

```
Matches: 10
```

```
Matching lines:
```

- The ocean covers more than seventy percent of the Earth's surface.
- Ocean currents help regulate the planet's climate and weather patterns.
- Marine biologists study ocean life, from tiny plankton to giant whales.
- The deepest part of the ocean is the Mariana Trench in the Pacific.
- Coral reefs are home to a huge variety of ocean species.
- Plastic pollution is a growing threat to ocean ecosystems worldwide.
- Ships and submarines allow humans to travel through and explore the ocean.
- Tides in the ocean are caused mainly by the gravitational pull of the moon.
- Ocean exploration remains limited, with much of the deep sea still unmapped.
- Fishing and shipping industries rely heavily on healthy ocean resources.

```
2. travel.txt
```

```
Matches: 10
```

```
Matching lines:
```

- Travel allows people to experience new cultures, food, and places firsthand.
- Budget travel focuses on finding affordable flights, stays, and local transport.
- Many travelers use guidebooks and apps to plan their trips in advance.
- Backpacking is a popular style of travel focused on light packing and flexibility.
- Travel broadens perspective by exposing people to different ways of life.
- Airports and train stations are major hubs for both domestic and international travel.
- Sustainable travel tries to minimize the environmental impact of tourism.
- Solo travel has become increasingly popular among younger generations.
- A good travel itinerary balances sightseeing with rest and free time.
- Travel insurance can help cover unexpected problems like delays or lost luggage.

```
Enter search keyword (or 'history' to view past searches, 'exit' to quit): |
```

Figure 15.2 - Searching for "ocean travel": lines matching either word are counted

15.3 Case-Insensitive Search and No-Match Query

```
Enter search keyword (or 'history' to view past searches, 'exit' to quit): HEALTH
Search Results
=====
1. health.txt
   Matches: 6
   Matching lines:
   - Good health depends on a balance of diet, exercise, sleep, and stress management.
   - Getting enough sleep each night is important for both physical and mental health.
   - Mental health is just as important as physical health for overall wellbeing.
   - Routine checkups with a doctor can catch health issues early.
   - Stress management techniques like meditation can improve long term health.
   - Small daily habits often have a bigger impact on health than occasional big changes.
2. ocean.txt
   Matches: 1
   Matching lines:
   - Fishing and shipping industries rely heavily on healthy ocean resources.
Enter search keyword (or 'history' to view past searches, 'exit' to quit): zzzznotfound
Search Results
=====
No matches found.
Enter search keyword (or 'history' to view past searches, 'exit' to quit): |
```

Figure 15.3 - "HEALTH" matches lowercase content; "zzzznotfound" returns no matches

15.4 Empty Input, Search History, and Exit

```
Enter search keyword (or 'history' to view past searches, 'exit' to quit):
Please enter a keyword.

Enter search keyword (or 'history' to view past searches, 'exit' to quit): history
Search History
=====
1. bitcoin
2. ocean travel
3. HEALTH
4. zzzznotfound

Enter search keyword (or 'history' to view past searches, 'exit' to quit): exit
Goodbye!

C:\Users\D Jai Deepak\Downloads\shopilot-FINAL\slab-browser-agent\New Folder>|
```

Figure 15.4 - Empty input is rejected, history lists all four prior queries, and exit ends the program